developed to ensure that decisions could be made consistently, even as systems grew more complex.

At the core of this system was a shared policy decision capability in the form of a common policy decision layer responsible for evaluating access decisions consistently across applications (building on the PARC model introduced in section 7.3). Rather than embedding access logic directly in application code, ACME's services increasingly relied on this common policy layer. Product teams created policies that aligned with business goals, while foundational policies, written by the identity and security teams, ensured consistent enforcement throughout the organization. Decisions were made at runtime, using relationships, attributes, and contextual signals to determine which actions could move forward (as explored in Chapter 12 on authorization context).

This shift changed how authority moved through ACME's platform. Relationship-based models enabled collaboration patterns that previously needed manual effort. Attribute-driven policies let high-security controls be applied dynamically rather than through strict process gates. Delegation mechanisms allowed both people and systems to act within clearly defined boundaries. Automation increased, but within decision frameworks that stayed visible and explainable.

Equally important was the governance that developed alongside the technology (see chapter 14 to read more about policy governance). ACME invested in defining ownership for decision signals, establishing expectations for freshness and assurance (building on the PIP model described in section 12.1), and creating review processes for policy changes. Decision logs became a source for operational insights (as discussed in section 17.3 on AI and policy analysis), allowing teams to understand not only what actions occurred but also why they were permitted or denied. This made it possible to identify inconsistencies, audit behavior, and improve policies based on real system outcomes. Over time, authorization moved from being an implementation detail to a managed capability that could be improved deliberately rather than rediscovered repeatedly.

As the decision surface expanded, testing and validation grew more complex and more critical. Decisions were no longer isolated in individual features or